

Campus Grievance Redressal System (CGRS)

*Project submitted in partial fulfilment of the requirements
for the award of the degree of*

MCA (Integrated)

Semester IV

January - May, 2026

Submitted by:
Somya Ranawat: 2111254

Under the guidance of:
Dr. Mohit Kumar Varma



School of Computer Science & IT
Devi Ahilya Vishwavidyalaya, Indore, (MP) - 452017, India

School of Computer Science & IT
Devi Ahilya Vishwavidyalaya, Indore, (MP) - 452017, India

DECLARATION

I hereby declare that the project titled “**Campus Grievance Redressal System**” submitted by me for the partial fulfilment of the requirement for the award of MCA (Integrated) to School of Computer Science & IT, Devi Ahilya Vishwavidyalaya, Indore, comprises my own work and due acknowledgement has been made in text to all other material used.

Signature of Faculty:

Date:

Place:

School of Computer Science & IT
Devi Ahilya Vishwavidyalaya, Indore, (MP) - 452017, India

CERTIFICATE FROM GUIDE

It is to certify that dissertation on “**Campus Grievance Redressal System**” submitted by Ms. Somya Ranawat: 2111254 to the School of Computer Science & IT, Devi Ahilya Vishwavidyalaya, Indore has been completed under my supervision and the work is carried out and presented in a manner required for its acceptance in partial fulfilment for the award of the degree of MCA (Integrated).

Project Guide

Signature:

Name:

Date:

School of Computer Science & IT
Devi Ahilya Vishwavidyalaya, Indore, (MP) - 452017, India

APPROVAL CERTIFICATE FROM EXAMINER(S)

It is to certify that we have examined the dissertation on “Campus Grievance Redressal System”, submitted by Ms. Somya Ranawat:2111254 to the School of Computer Science & IT, Devi Ahilya Vishwavidyalaya, Indore and hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in partial fulfilment for the award of the degree of MCA(Integrated).

Internal Examiner

Signature:

Name:

Date:

External Examiner

Signature:

Name:

Date:

School of Computer Science & IT
Devi Ahilya Vishwavidyalaya, Indore, (MP) - 452017, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our respected guide Dr. Mohit Kumar Varma for his valuable guidance, continuous support, and encouragement throughout the development of this project. His insights and suggestions greatly contributed to the successful completion of this work.

We are also thankful to the faculty members of the School of Computer Science & IT, Devi Ahilya Vishwavidyalaya, Indore, for providing us with the necessary knowledge, resources, and support required during the course of this project.

We would like to extend our heartfelt thanks to our friends and classmates for their cooperation, constructive feedback, and motivation at various stages of the project.

Finally, we are deeply grateful to our family members for their constant encouragement, understanding, and moral support, which helped us complete this project successfully.

Table of Contents

S. No.	Chapter Name	Page No.
	List of Figures	vii
	List of Tables	viii
	List of Algorithms	ix
1.	INTRODUCTION	1
	1.1 Background	1
	1.2 Purpose of the Project	1
	1.3 Project Scope	2
	1.4 Project Objectives	2
2.	PROJECT PLANNING AND SCHEDULING	3
	2.1 Project Plan	3
	2.2 Work Breakdown Structure	4
	2.3 Gantt Chart	5
	2.4 PERT Chart/ CPM	5
	2.5 Team Structure and Responsibilities	6
	2.6 Project Development Methodology	6
	2.7 Hardware and Software Requirements	7
3.	SYSTEM ANALYSIS	8
	3.1 Problem Description	8
	3.1.1 Problem Definition	8
	3.1.2 Proposed Solution	8
	3.2 Requirements	9
	3.2.1 Functional Requirements	9
	3.2.2 Non-Functional Requirements	9
	3.3 Problem Analysis Diagrams	9
	3.3.1 Data Flow Diagram/ Process Flow Diagram	10
	3.3.2 Use Case Diagram and Sequence Diagram	11
	3.4 Database Schema	12
4.	SYSTEM DESIGN	13
	4.1 System Architecture	13
	4.2 Physical Design	14
	4.2.1 Structure Chart	14
	4.2.2 ER Diagram	15
	4.2.3 Class Diagram and Object Diagram	15
	4.3 Input and Output Design	16
	4.4 Algorithmic Design	17
5.	IMPLEMENTATION	19
	5.1 Source Code	19
	5.2 Integration of Modules/ Files	22
	5.3 Screenshots/ Reports/ Dataset details (if required)	23
6.	SYSTEM TESTING	28
	6.1 Test Case Design	28
	6.1.1 Unit Testing	28
	6.1.2 Integration Testing	29
	6.1.3 System Testing	29
	6.1.4 Acceptance Testing	30

	6.2 Specific System Testing	30
	6.3 Test Reports	31
7.	CONCLUSION OF THE PROJECT	32
	7.1 Results	32
	7.2 Conclusion	33
	7.3 Limitations of the project	33
	7.4 Future Work	34
	7.5 Lessons Learned	34
	REFERENCES	36
	APPENDICES	37

List of Figures

S. No.	Figure Name	Page No.
1.	Work Breakdown Structure	4
2.	Gantt Chart	5
3.	PERT Chart / CPM	5
4.	DFD Level 0	10
5.	DFD Level 1	10
6.	DFD Level 2	11
7.	Use Case Diagram	11
8.	Sequence Diagram	12
9.	System Architecture	13
10.	Structure Chart	14
11.	ER Diagram	15
12.	Class Diagram	15
13.	Object Diagram	15
14.	Login Page	23
15.	User Registration Page	23
16.	Grievance Submission Page	24
17.	Grievance Details Submission Page	24
18.	Admin Dashboard	25
19.	Admin Grievance Management Page	26
20.	Grievance Status Update Page	26
21.	Report & Analytics Dashboard	27

List of Tables

S. No.	Table Name	Page No.
1.	Database Schema	12
2.	Unit Testing	28
3.	Integration Testing	29
4.	System Testing	29
5.	Acceptance Testing	30
6.	Specific System Testing	30

List of Algorithms

S. No.	Algorithm Name	Page No.
1.	Algorithm for User Authentication	17
2.	Algorithm for Grievance Submission	17
3.	Algorithm for Grievance Management	18
4.	Algorithm for Grievance Tracking	18

ABSTRACT

The **Campus Grievance Redressal System (CGRS)** is a web-based application designed to provide an efficient and transparent platform for faculties to register and track their grievances. In many educational institutions, complaints are often handled manually, leading to delays, lack of accountability, and poor communication between faculties and administration.

This project aims to digitalize the grievance handling process by enabling faculties to submit complaints online, which are then reviewed and managed by the appropriate administrative authority. The system allows real-time tracking of grievance status, ensuring transparency and timely resolution of complaints.

The system is developed using modern technologies such as Java (Spring Boot), MySQL, and web-based frontend technologies such as HTML, CSS, and Thymeleaf, ensuring scalability, security, and performance. Key features include secure user authentication, online grievance submission, complaint status tracking, admin dashboard management, downloadable grievance reports, and automated email notifications for complaint submission and status updates.

This system significantly improves communication between faculties and administration, reduces grievance resolution time, and maintains proper digital records for future reference. Future enhancements may include mobile application integration, push notifications, multilingual support, and advanced analytics dashboards.

CHAPTER 1

INTRODUCTION

1.1 Background

In educational institutions, faculties often encounter various issues related to infrastructure, academics, administration, and campus facilities. Traditionally, grievance handling systems rely on manual processes such as written complaints or verbal communication, which are time-consuming, inefficient, and lack proper tracking mechanisms. This often results in delayed responses, miscommunication, and lack of accountability.

With the advancement of technology, there is a growing need for a digital solution that can streamline the grievance handling process. A web-based system can provide a structured platform for submitting, managing, and resolving complaints efficiently. The *Campus Grievance Redressal System (CGRS)* is designed to address these challenges by offering a transparent, accessible, and user-friendly interface for both faculties and administrators.

1.2 Purpose of the Project

The main purpose of this project is to develop a centralized and automated system for managing faculty grievance within a campus. The system aims to simplify the complaint submission process and ensure that all grievances are addressed in a timely and efficient manner.

It provides a platform where users can:

- Submit grievances online
- Track the status of their complaints
- Receive timely updates regarding their grievances
- Ensure transparency in the resolution process

Additionally, the system enables administrators to manage, categorize, and resolve grievances effectively. The project focuses on improving communication between faculties and the administration while maintaining transparency, accountability, and efficiency.

1.3 Project Scope

The scope of the *Campus Grievance Redressal System* is limited to managing grievances within an educational institution. The system is designed to handle complaints related to various categories such as infrastructure, academics, library, and other campus services.

The application is developed as a web-based system, making it accessible through standard web browsers. It includes features such as user authentication, complaint submission, status tracking, and administrative control.

The system is designed for use within a college or university campus and includes:

- Faculty login and complaint submission
- Admin panel for managing grievances
- Categorization of complaints
- Complaint status tracking system

Furthermore, the system is scalable and can be extended in the future to include additional features such as mobile applications, notification services, and advanced analytics.

1.4 Project Objectives

The key objectives of the project are:

- To develop a digital platform for grievance submission and management
- To reduce the time required for complaint resolution
- To ensure transparency and accountability in the grievance process
- To provide real-time tracking of complaint status
- To maintain a structured database of all grievances
- To improve communication between faculty and administration
- To enhance overall efficiency in handling campus-related issues

CHAPTER 2

PROJECT PLANNING AND SCHEDULING

2.1 Project Plan

The project plan for the *Campus Grievance Redressal System (CGRS)* was developed to ensure systematic execution and timely completion of all project activities. The project was carried out in a structured manner by dividing it into multiple phases, each focusing on a specific aspect of development.

Initially, the **requirement analysis phase** was conducted to understand the problems in the existing manual grievance system and to identify user needs. This was followed by the **system design phase**, where the architecture, database schema, and overall workflow of the system were planned.

In the **development phase**, the application was implemented using appropriate technologies such as Java (Spring Boot) for backend development and MySQL for database management. The system modules, including user management, grievance handling, and admin functionalities, were developed and integrated.

After development, the project moved to the **testing phase**, where different types of testing such as unit testing and system testing were performed to ensure the correctness and reliability of the system.

Finally, in the **deployment phase**, the system was made ready for use, ensuring that all functionalities were working properly and efficiently.

The project plan ensured proper time management, resource utilization, and coordination between team members, leading to the successful completion of the project.

2.2 Work Breakdown Structure

Campus Grievance Redressal System (CGRS)

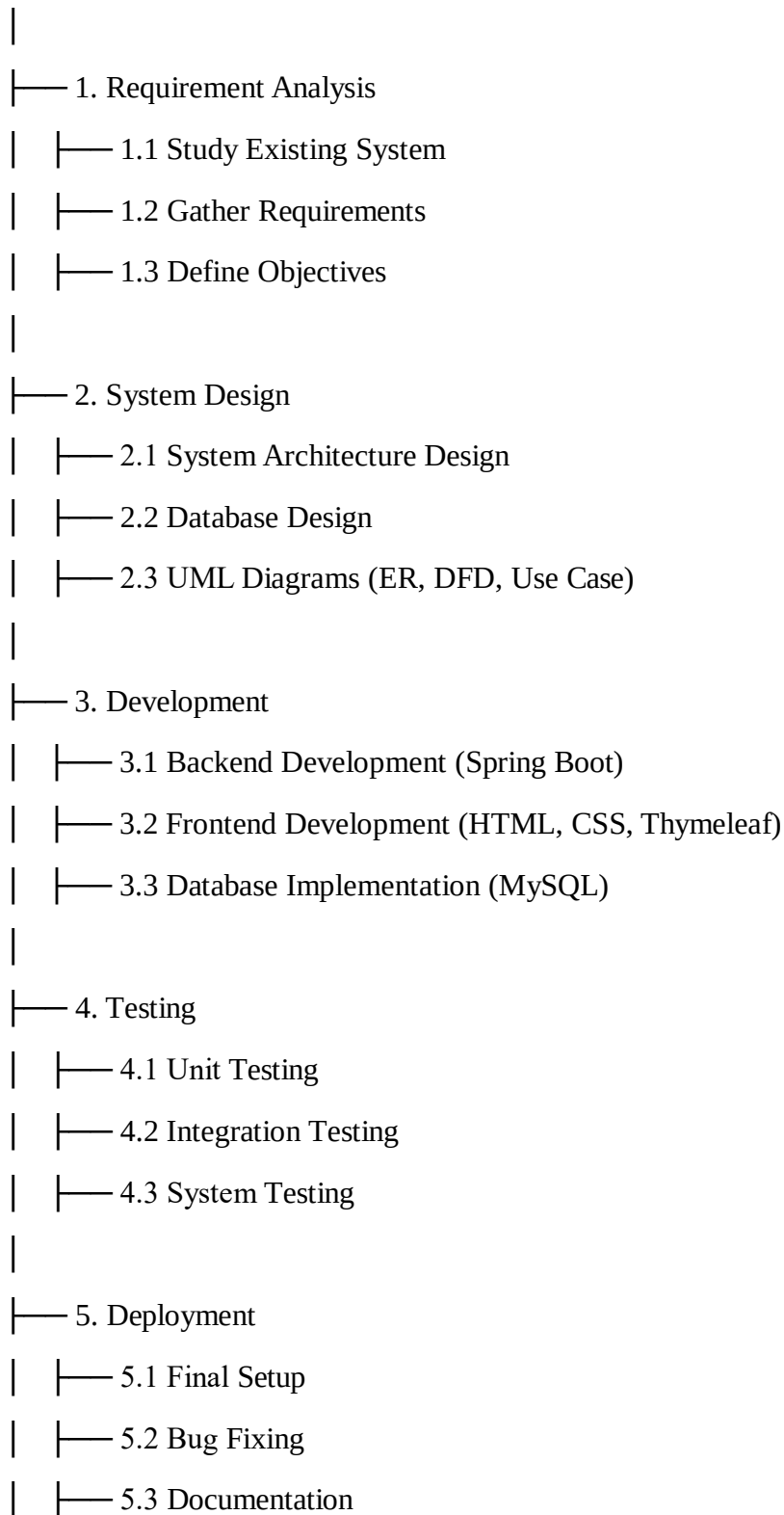


Fig 1. Work Breakdown Structure

2.3 Gantt Chart

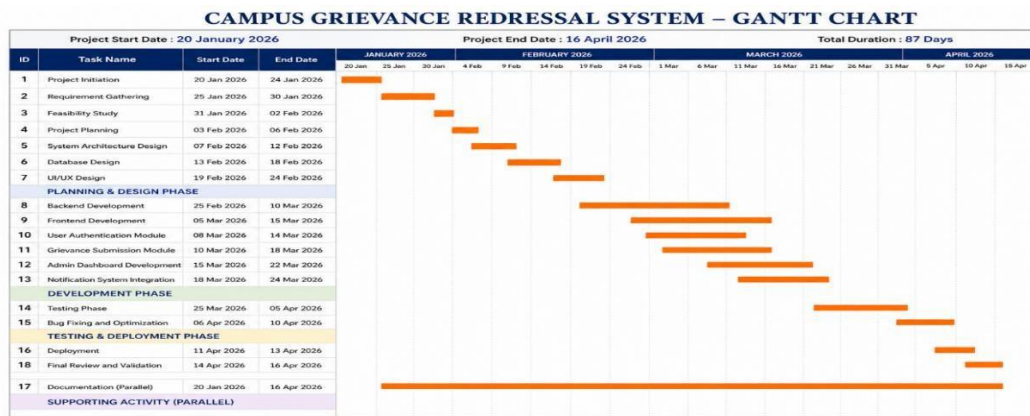


Fig 2. Gantt Chart

2.4 PERT Chart/ CPM

Activity	Task	Duration (Weeks)
A	Requirement Analysis	1 week
B	System Design	1 week
C	Database Design	2 weeks
D	Backend Development	3 weeks
E	Frontend Development	2 weeks
F	System Testing & Integration	2 weeks
G	Deployment & Documentation	1 week

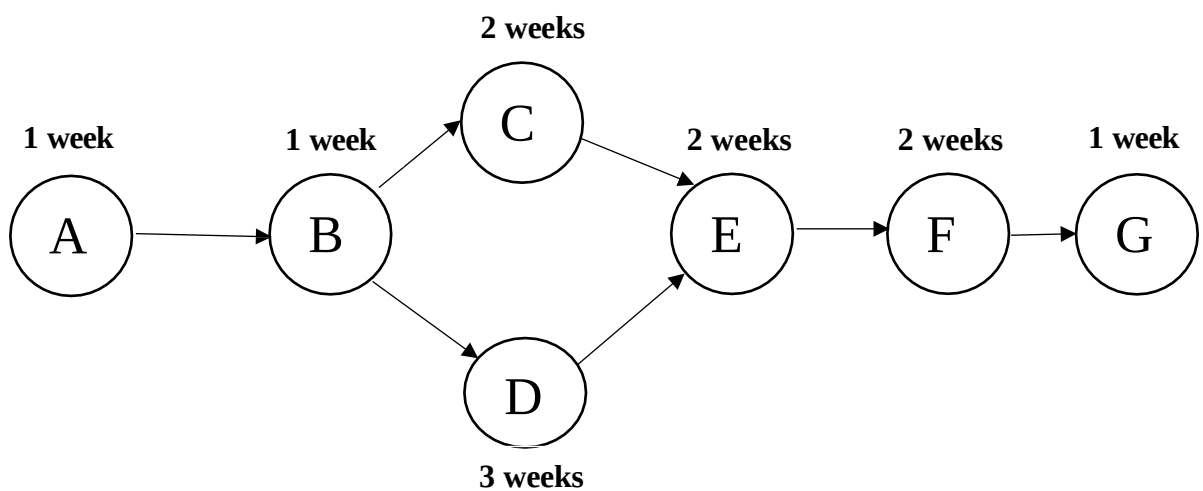


Fig 3. PERT Chart/ CPM

Critical Path:

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

(because D takes more time than C)

1 week + 1 week + 3 weeks + 2 week + 2 week + 1 week = 10 weeks

2.5 Team Structure and Responsibilities

The project was developed by a team of two members, with clearly defined roles and responsibilities to ensure efficient execution and timely completion of all tasks.

- **a. Team Member 1**

Responsible for backend development, including designing and implementing REST APIs using Spring Boot. Also handled database design and integration using MySQL, ensuring proper data management and system functionality.

- **b. Team Member 2**

Responsible for frontend development, including designing user interfaces using HTML, CSS and Thymeleaf. Also contributed to system testing and validation to ensure a user-friendly experience.

Both team members collaboratively worked on requirement analysis, system design, testing, and documentation. Effective coordination and communication between the team members helped in successfully completing the project within the given timeframe.

2.6 Project Development Methodology

The development of the *Campus Grievance Redressal System (CGRS)* follows the **Agile Development Methodology**, which is an iterative and flexible approach to software development. Agile methodology focuses on continuous improvement, regular feedback, and incremental delivery of the system.

In this project, the development process was divided into small phases (iterations), where each phase included planning, design, development, and testing. This allowed the team to identify issues early and make necessary improvements at each stage.

The key features of Agile methodology include:

- **Iterative Development:** The system is developed in small modules rather than all at once
- **Flexibility:** Changes can be easily incorporated during development
- **Continuous Testing:** Regular testing ensures early detection of errors
- **User Feedback:** Feedback is considered to improve system functionality

Using Agile methodology helped in better coordination between team members, improved system quality, and ensured timely completion of the project.

2.7 Hardware and Software Requirements

The development and execution of the *Campus Grievance Redressal System (CGRS)* require certain hardware and software components to ensure smooth functioning and performance.

Hardware Requirements

The system does not require high-end hardware and can run efficiently on standard computing devices. The basic hardware requirements are:

- Computer/Laptop
- Minimum 4 GB RAM (8 GB recommended for better performance)
- Processor: Intel i3 or above
- Hard Disk: Minimum 500 GB Hard Disk or 256 GB SSD
- Internet Connection for system access and email notification services

Software Requirements

The system is developed using modern technologies and tools. The software requirements are:

- **Operating System:** Windows
- **Programming Language:** Java
- **Framework:** Spring Boot
- **Frontend Technologies:** HTML, CSS
- **Template Engine:** Thymeleaf
- **Database:** MySQL
- **IDE:** IntelliJ IDEA
- **Web Browser:** Google Chrome / Microsoft Edge

CHAPTER 3

SYSTEM ANALYSIS

3.1 Problem Description

In educational institutions, faculties frequently face issues related to infrastructure, academics, administration, and campus facilities. However, the existing grievance handling systems are mostly manual, relying on written applications or verbal communication. These methods are inefficient, time-consuming, and lack proper tracking mechanisms.

Due to the absence of a centralized system, complaints may get delayed, lost, or ignored. Faculties are often unable to track the progress of their grievances, leading to dissatisfaction and lack of trust in the system. Additionally, there is limited transparency and accountability in resolving complaints.

Therefore, there is a need for a digital system that can streamline the grievance handling process, ensure timely resolution, and improve communication between faculties and administration.

3.1.1 Problem Definition

The main problem is the lack of an efficient and transparent system for managing faculty grievances. The existing manual process does not provide proper tracking, timely updates, or accountability, resulting in delays and inefficiencies in resolving complaints.

3.1.2 Proposed Solution

To overcome the limitations of the existing system, a web-based *Campus Grievance Redressal System (CGRS)* is proposed. This system allows faculties to submit complaints online and track their status in real-time.

The system provides a centralized platform where grievances are categorized and assigned to the appropriate authority. Administrators can review, update, and resolve complaints efficiently. The system ensures transparency, accountability, and faster resolution of issues.

3.2 Requirements

The requirements define the functionalities and performance criteria that the system must fulfill to operate effectively.

3.2.1 Functional Requirements

The functional requirements describe the core features of the system:

- User registration and login functionality
- Submission of grievances by faculties
- Categorization of complaints
- Viewing and tracking complaint status
- Admin dashboard for managing complaints
- Updating grievance status (Pending, In Progress, Resolved)
- Managing categories of complaints
- Sending notifications or updates to users

3.2.2 Non-Functional Requirements

The non-functional requirements define the quality attributes of the system:

- **Performance:** The system should respond quickly to user requests
- **Security:** User data must be protected with proper authentication and authorization
- **Reliability:** The system should function without failure
- **Usability:** The interface should be user-friendly and easy to navigate
- **Scalability:** The system should handle increasing number of users
- **Availability:** The system should be accessible at all times

3.3 Problem Analysis Diagrams

Problem analysis diagrams are used to visually represent the structure, flow of data, and interactions within the *Campus Grievance Redressal System (CGRS)*. These diagrams help in understanding the functioning of the system and provide a clear representation of processes involved in grievance management.

3.3.1 Data Flow Diagram/ Process Flow Diagram

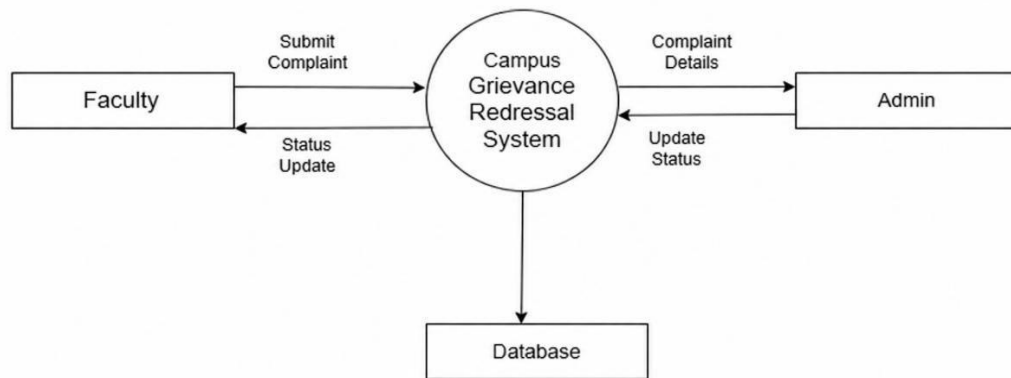


Fig 4. DFD LEVEL 0

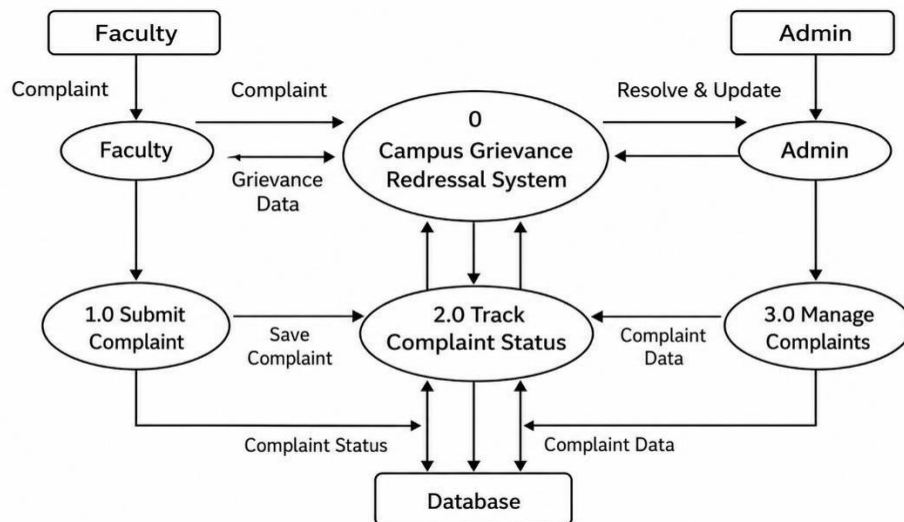


Fig 5. DFD LEVEL 1

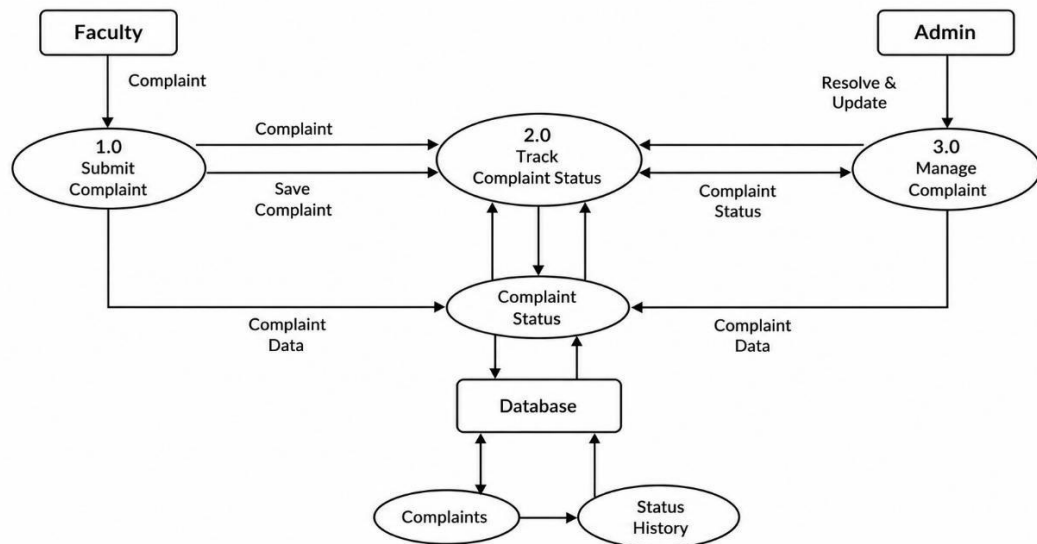


Fig 6. DFD LEVEL 2

3.3.2 Use Case Diagram and Sequence Diagram

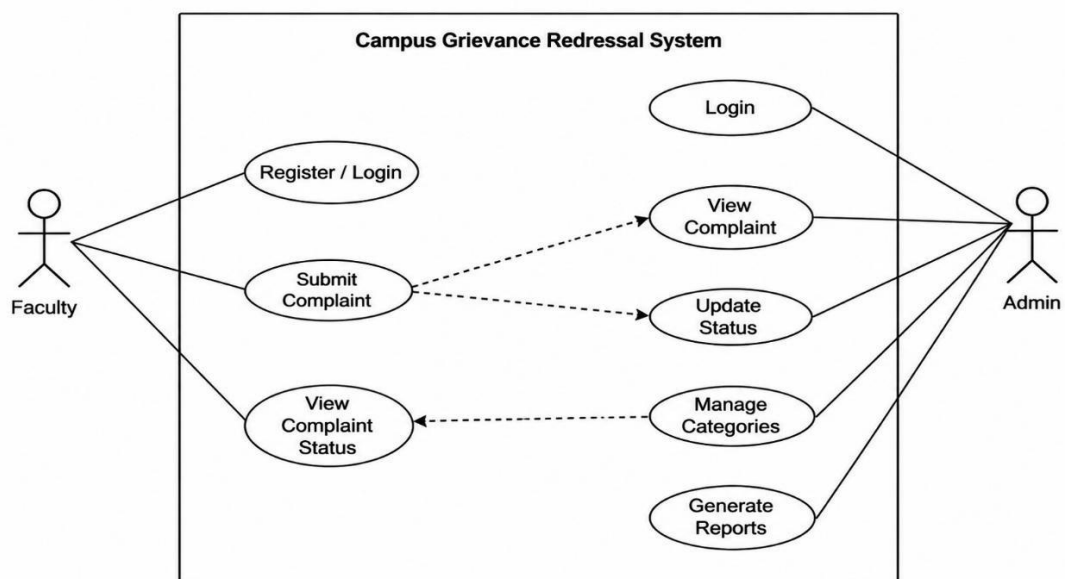


Fig 7. Use Case Diagram

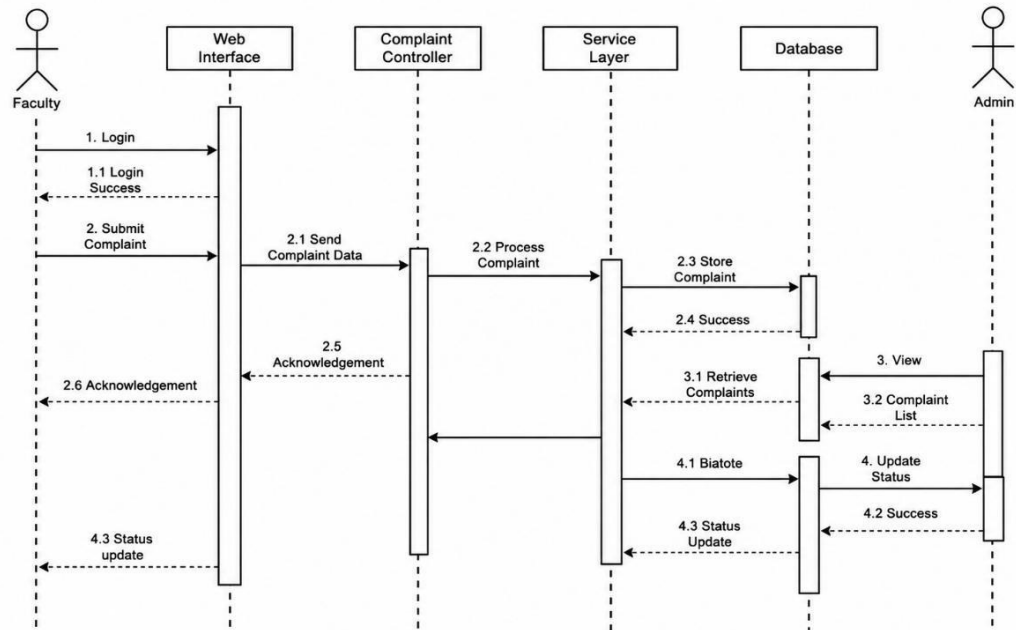


Fig 8. Sequence Diagram

3.4 Database Schema

USERS			
Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique user ID
name	VARCHAR(40)	NOT NULL	User full name
email	VARCHAR(40)	UNIQUE, NOT NULL	User email
password	VARCHAR(255)	NOT NULL	User Password
role	ENUM('USER', 'ADMIN')	NOT NULL	Role of user
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation datetime

GRIEVANCES			
Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique grievance ID
description	TEXT	NOT NULL	Grievance details
status	ENUM('PENDING', 'IN_PROGRESS', 'RESOLVED', 'REJECTED')	NOT NULL	Current status
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Grievance submission time
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last updated time
user_id	INT	FK -> USERS(id) ON DELETE CASCADE	Submitted by user
category_id	INT	FK -> CATEGORIES(id) ON DELETE RESTRICT	Category of grievance
assigned_id	INT	FK -> USERS(id) NULL	Assign assigned (optional)

Table 1. Database Schema

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

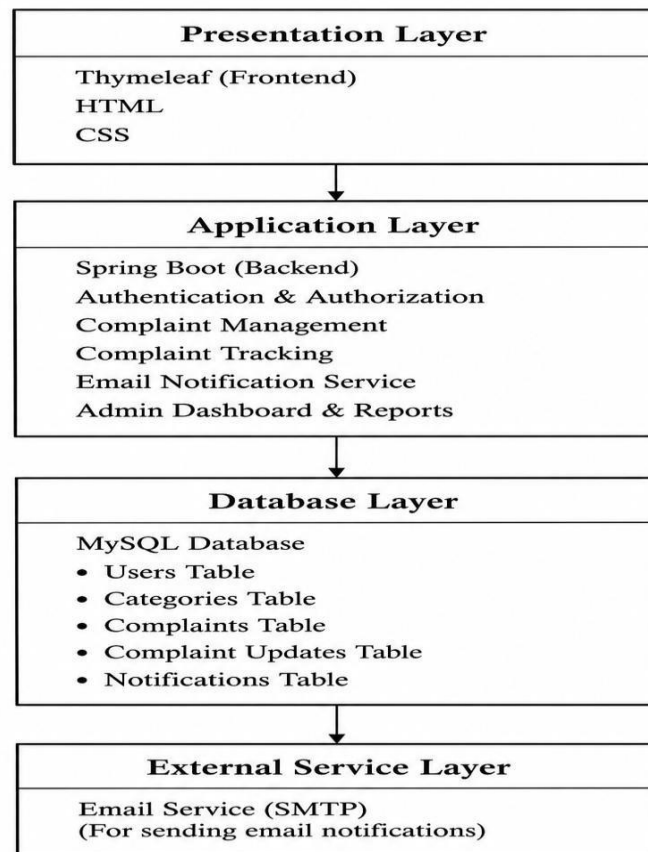


Fig 9. System Architecture

Working:

- The system allows faculties to register/login securely and submit grievances through a web interface developed using Thymeleaf. Complaint details are processed by the Spring Boot backend and stored in the MySQL database.
- The Admin accesses the dashboard to view submitted complaints, review details, update complaint status, and provide remarks or resolutions. The system maintains complaint records and updates status in real time for transparent grievance tracking.
- Additionally, email notifications are sent through the SMTP service whenever a complaint is submitted or its status is updated. This keeps faculties informed about grievance progress and improves communication between faculties and administration.

4.2 Physical Design

- The **Physical Design** of the **Campus Grievance Redressal System (CGRS)** describes the actual implementation of hardware and software components required to run the system efficiently.
- The system is developed as a web-based application that can be accessed through a computer or laptop connected to the internet. Faculties and administrators interact with the system using a web browser interface developed with Thymeleaf, HTML, and CSS.
- The application logic is implemented using Spring Boot, which handles authentication, complaint submission, complaint tracking, admin dashboard operations, and email notification services.
- The backend is connected to a MySQL database server that stores user details, complaint records, complaint updates, categories, and notifications securely for efficient retrieval and management.
- The system also integrates SMTP email services for sending automated notifications related to complaint submission, status updates, and grievance resolution.
- The physical deployment consists of:
 1. Client System (Faculty/Admin Computer with Web Browser)
 2. Application Server (Spring Boot Backend)
 3. Database Server (MySQL)
 4. Email Service Server (SMTP)

This physical design ensures secure access, reliable data storage, and smooth communication between faculties and administrators.

4.2.1 Structure Chart

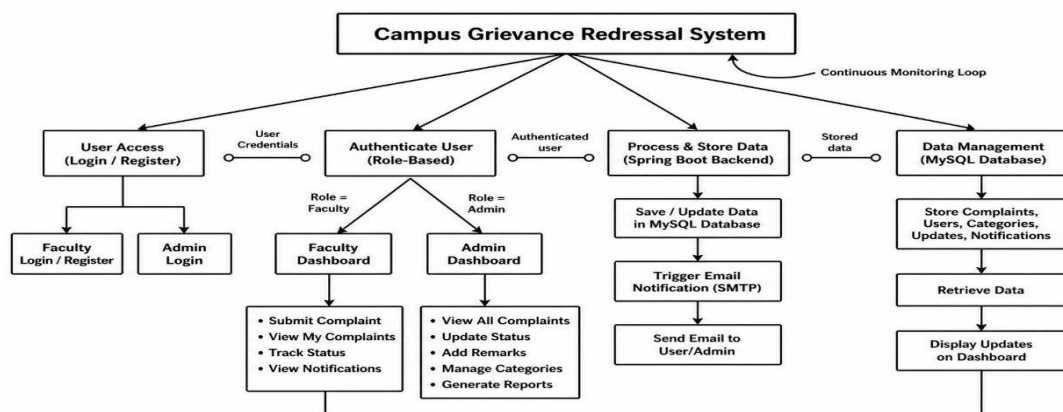


Fig 10. Structure Chart

4.2.2 ER Diagram

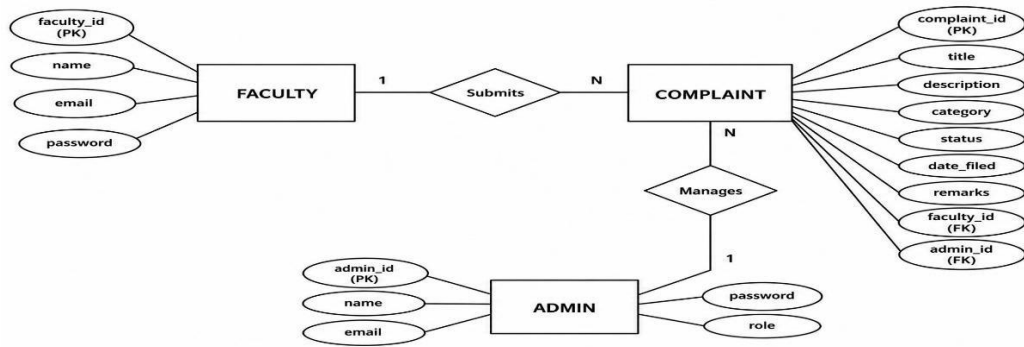


Fig 11. ER Diagram

4.2.3 Class Diagram and Object Diagram

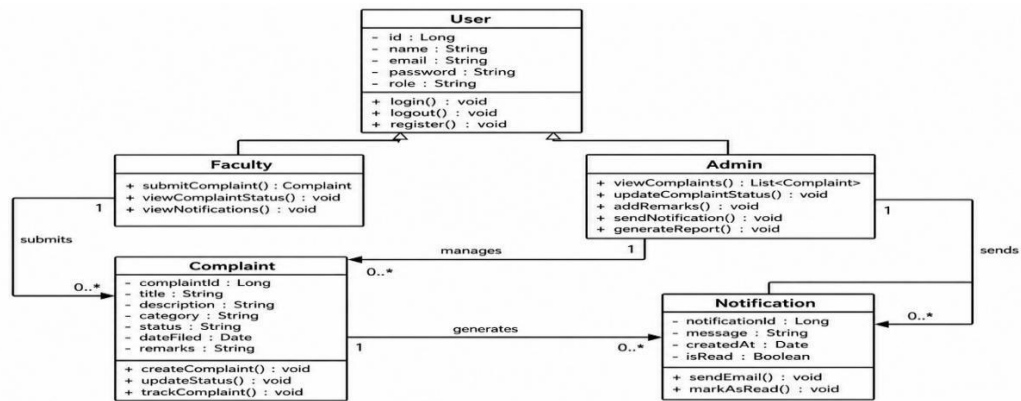


Fig 12. Class Diagram

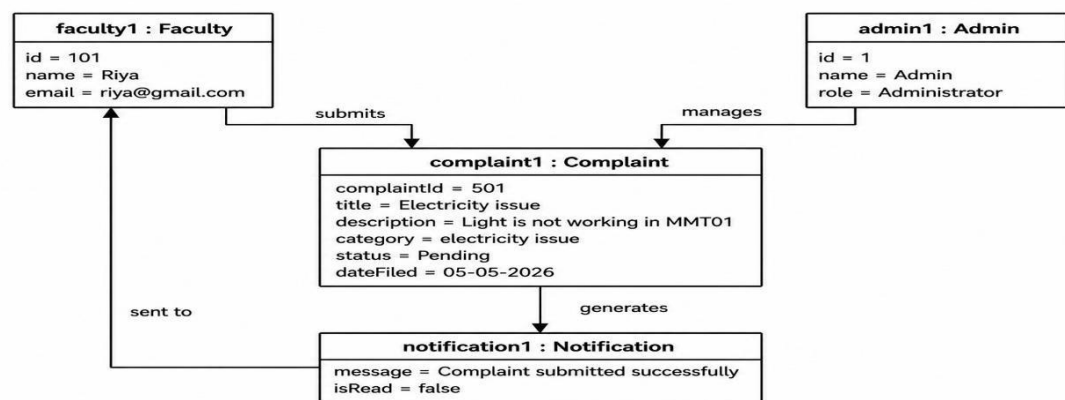


Fig 13. Object Diagram

4.3 Input and Output Design

Input Design

Input design focuses on collecting accurate and relevant data from users in a simple and user-friendly manner. In the **Campus Grievance Redressal System**, different input forms are designed for faculties and administrators.

Faculty Input:

- Faculty Registration Form
- Name
- Email ID
- Password
- Login Form
- Email ID
- Password
- Complaint Submission Form
- Complaint Title
- Description
- Category
- Supporting Image/Proof Upload (if applicable)

Admin Input:

- Admin Login Form
- Email ID
- Password
- Complaint Management Form
- Update Complaint Status
- Add Remarks/Comments

This ensures that only correct and complete information is stored in the system.

Output Design

Output design focuses on presenting processed information clearly and effectively to users.

The system generates the following outputs:

Faculty Output:

- Registration confirmation message
- Login success/failure notification
- Complaint submission confirmation

- Complaint status tracking (Pending / In Progress / Resolved / Rejected)
- Complaint history report
- Email notification for status updates

Admin Output:

- Dashboard displaying total complaints
- Category-wise complaint summary
- Pending complaint list
- Complaint details with uploaded proof
- Status update confirmation
- Report of resolved complaints

The output screens are designed to be clear, responsive, and easy to understand for both Facultys and administrators.

4.4 Algorithmic Design

The system follows a structured algorithm for handling grievances efficiently.

Algorithm: Campus Grievance Redressal System

4.4.1 Algorithm for User Authentication

1. Start the system.
2. Display login/registration page.
3. Accept user details (Email, Password).
4. Validate entered credentials.
5. If valid, redirect user to dashboard.
6. Else, display invalid login message.
7. End.

4.4.2 Algorithm for Grievance Submission

1. Start the system.
2. Faculty logs in.
3. Open grievance submission form.
4. Enter complaint details and category.
5. Upload proof/image (optional).
6. Validate details.
7. Save complaint in database with Pending status.

8. Send confirmation email notification.
9. Display successful submission message.
10. End.

4.4.3 Algorithm for Complaint Management

1. Start the system.
2. Admin logs in.
3. View submitted grievances.
4. Review complaint details.
5. Update status (Pending/In Progress/Resolved/Rejected).
6. Add remarks/comments.
7. Save updates in database.
8. Send status update email notification.
9. End.

4.4.4 Algorithm for Complaint Tracking

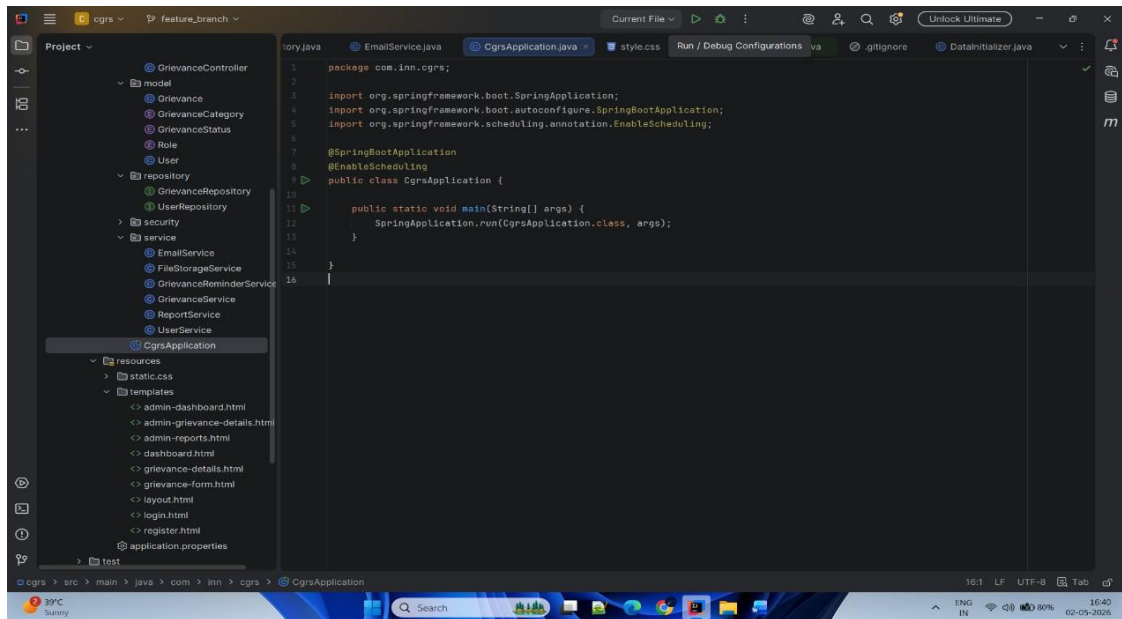
1. Start the system.
2. Faculty logs in.
3. Open grievance dashboard.
4. Fetch submitted complaints.
5. Display complaint status and remarks.
6. Update status whenever changes occur.
7. End.

CHAPTER 5

IMPLEMENTATION

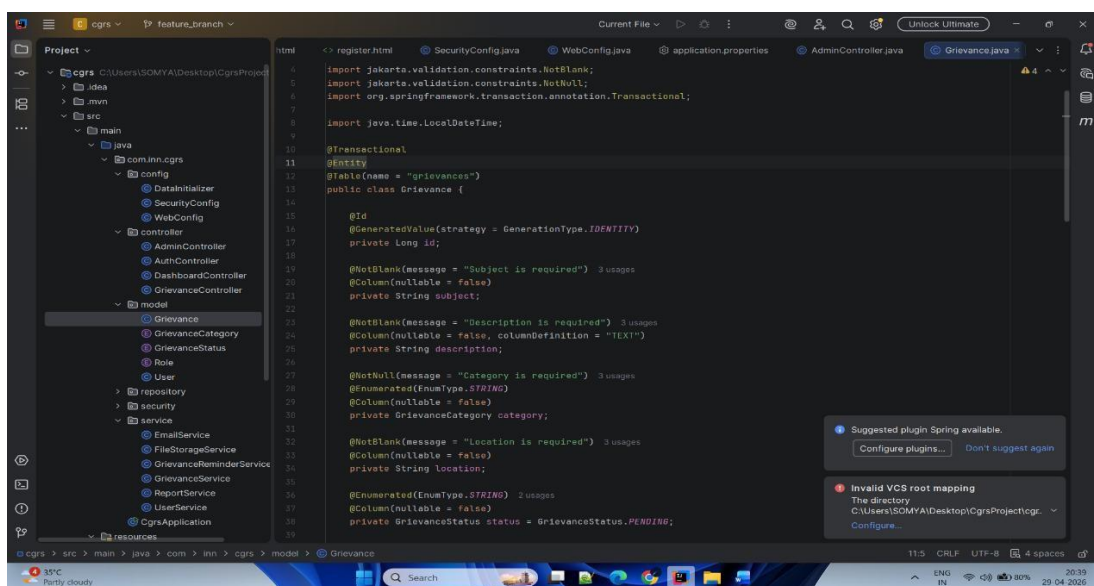
5.1 Source Code

5.1.1 Application Entry Point



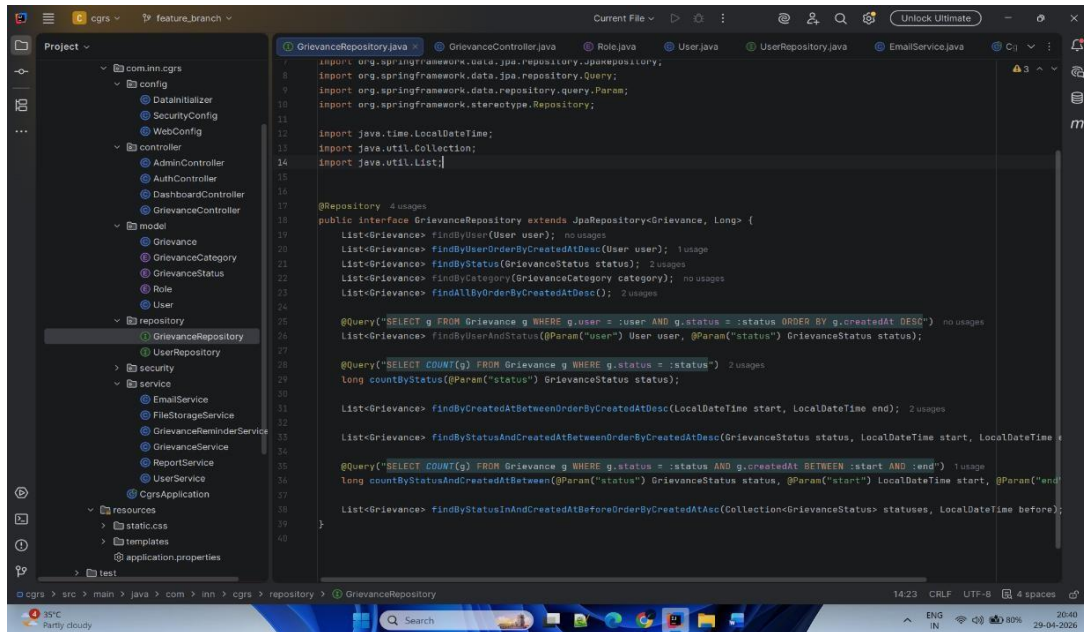
5.1.2 Entity Model Class

(Grievance.java / User.java)



5.1.3 Repository Layer

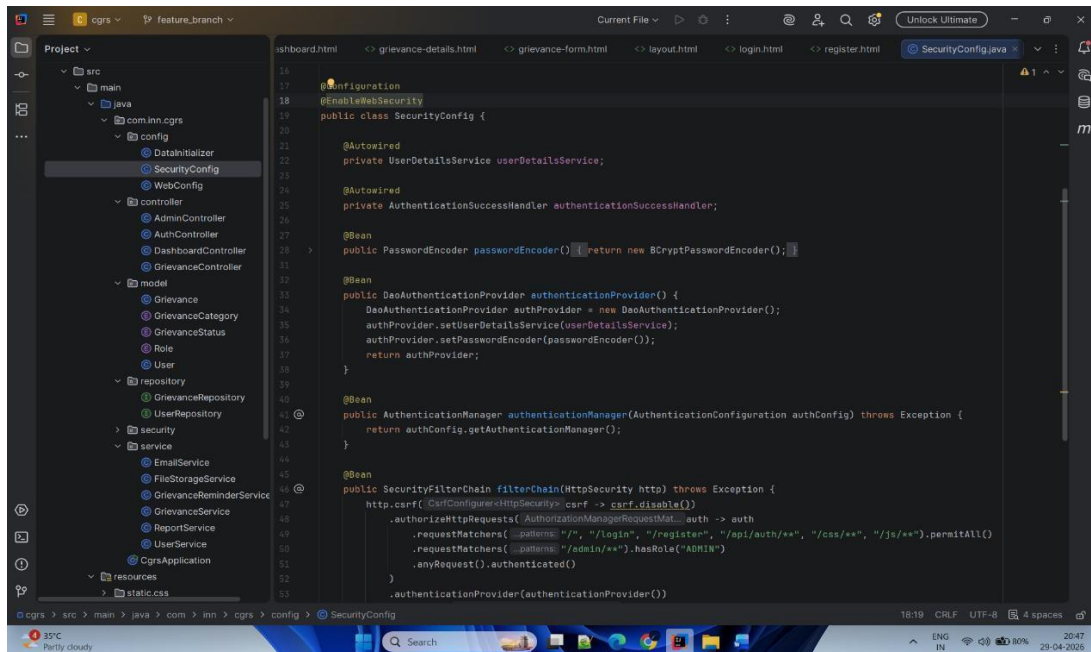
(GrievanceRepository.java)



```
7 import org.springframework.data.jpa.repository.JpaRepository;
8 import org.springframework.data.jpa.repository.Query;
9 import org.springframework.data.repository.query.Param;
10 import org.springframework.stereotype.Repository;
11
12 import java.time.LocalDateTime;
13 import java.util.Collection;
14 import java.util.List;
15
16 @Repository
17 public interface GrievanceRepository extends JpaRepository<Grievance, Long> {
18     List<Grievance> findByIds(Long... ids);
19     List<Grievance> findById(Long id);
20     List<Grievance> findByUserOrderByIdAsc(User user);
21     List<Grievance> findByStatus(GrievanceStatus status);
22     List<Grievance> findByCategory(GrievanceCategory category);
23     List<Grievance> findAllByOrderByIdAsc();
24
25     @Query("SELECT g FROM Grievance g WHERE g.user = :user AND g.status = :status ORDER BY g.createdAt DESC")
26     List<Grievance> findByUserAndStatus(@Param("user") User user, @Param("status") GrievanceStatus status);
27
28     @Query("SELECT COUNT(g) FROM Grievance g WHERE g.status = :status")
29     Long countByStatus(@Param("status") GrievanceStatus status);
30
31     List<Grievance> findByCreatedAtBetween(LocalDateTime start, LocalDateTime end);
32
33     List<Grievance> findByStatusAndCreatedAtBetween(GrievanceStatus status, LocalDateTime start, LocalDateTime end);
34
35     @Query("SELECT COUNT(g) FROM Grievance g WHERE g.status = :status AND g.createdAt BETWEEN :start AND :end")
36     Long countByStatusAndCreatedAtBetween(@Param("status") GrievanceStatus status, @Param("start") LocalDateTime start, @Param("end") LocalDateTime end);
37
38     List<Grievance> findByStatusInAndCreatedAtBefore(Collection<GrievanceStatus> statuses, LocalDateTime before);
39 }
```

5.1.4 Security Configuration Module

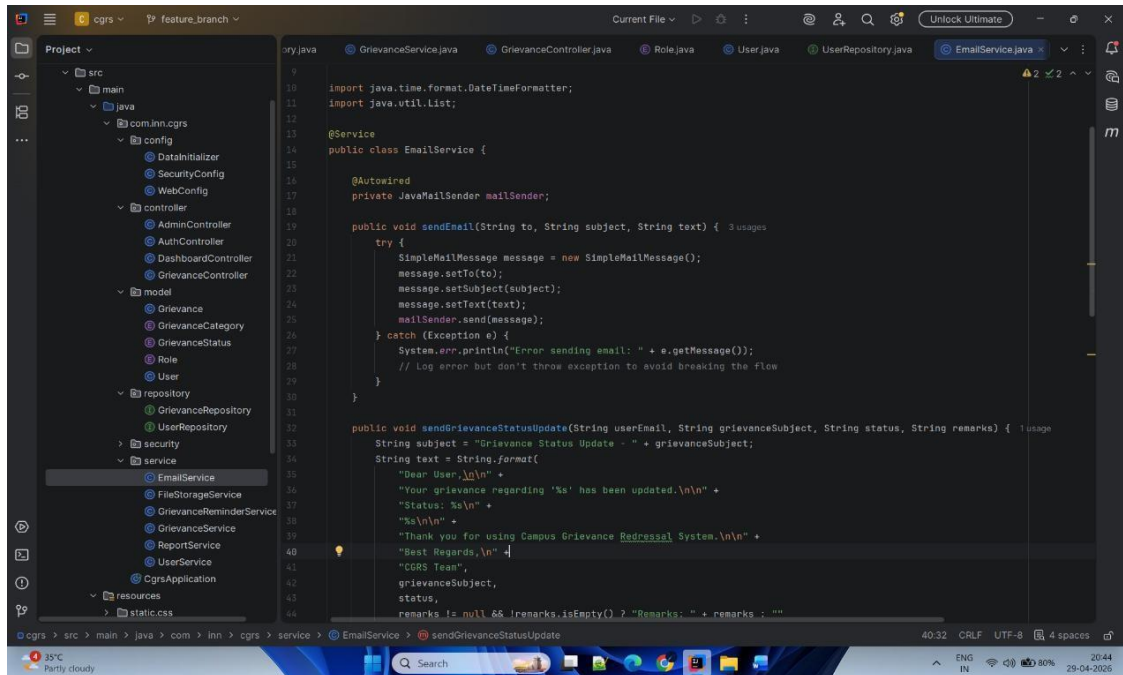
(SecurityConfig.java)



```
16 @Configuration
17 @EnableWebSecurity
18 public class SecurityConfig {
19
20     @Autowired
21     private UserDetailsServiceImpl userDetailsService;
22
23     @Autowired
24     private AuthenticationSuccessHandler authenticationSuccessHandler;
25
26     @Bean
27     public PasswordEncoder passwordEncoder() {
28         return new BCryptPasswordEncoder();
29     }
30
31     @Bean
32     public DaoAuthenticationProvider authenticationProvider() {
33         DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
34         authProvider.setUserDetailsService(userDetailsService);
35         authProvider.setPasswordEncoder(passwordEncoder());
36         return authProvider;
37     }
38
39     @Bean
40     public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig) throws Exception {
41         return authConfig.getAuthenticationManager();
42     }
43
44     @Bean
45     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
46         http.csrf((CsrfConfigurer<HttpSecurity> csrf) -> csrf.disable())
47             .authorizeHttpRequests((AuthorizationManagerRequestMatcher<?> auth) -> auth
48                 .requestMatchers(pattern: "/", "/login", "/register", "/api/auth/**", "/css/**", "/js/**").permitAll()
49                 .requestMatchers(pattern: "/admin/**").hasRole("ADMIN")
50                 .anyRequest().authenticated()
51             )
52             .authenticationProvider(authenticationProvider());
53     }
54 }
```

5.1.5 Email Notification Module

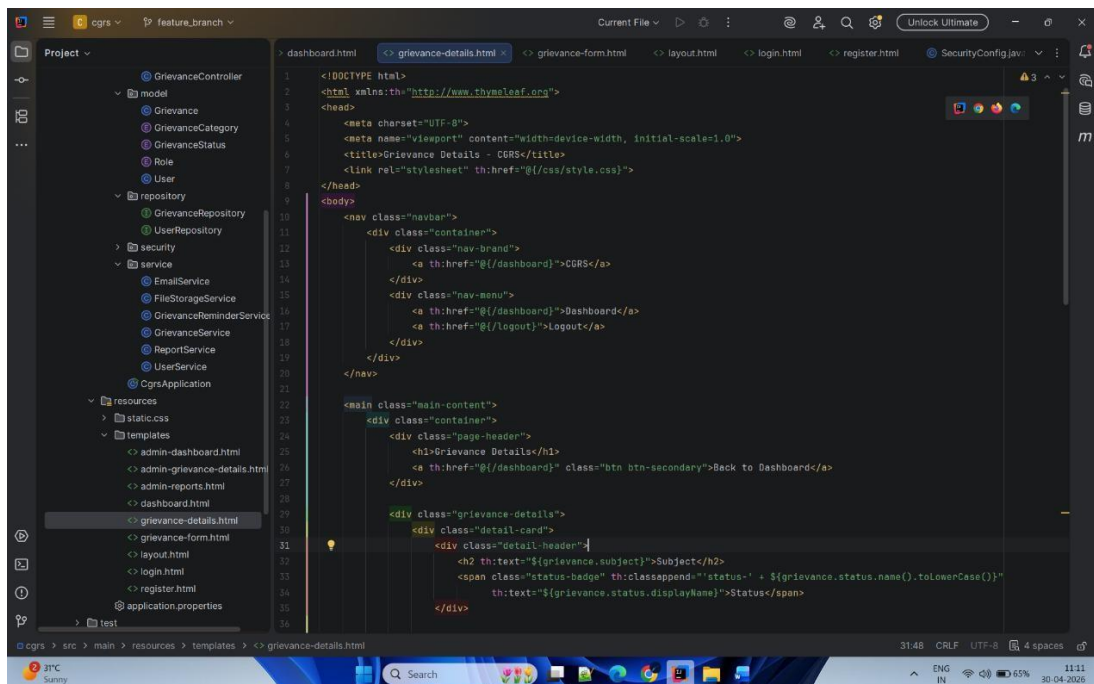
(EmailService.java)



```
1  import java.time.format.DateTimeFormatter;
2  import java.util.List;
3
4  @Service
5  public class EmailService {
6
7      @Autowired
8      private JavaMailSender mailSender;
9
10     public void sendEmail(String to, String subject, String text) {
11         try {
12             SimpleMailMessage message = new SimpleMailMessage();
13             message.setTo(to);
14             message.setSubject(subject);
15             message.setText(text);
16             mailSender.send(message);
17         } catch (Exception e) {
18             System.err.println("Error sending email: " + e.getMessage());
19             // Log error but don't throw exception to avoid breaking the flow
20         }
21     }
22
23     public void sendGrievanceStatusUpdate(String userEmail, String grievanceSubject, String status, String remarks) {
24         String subject = "Grievance Status Update - " + grievanceSubject;
25         String text = String.format(
26             "Dear User, \n\n" +
27             "Your grievance regarding '%s' has been updated. \n\n" +
28             "Status: %s \n\n" +
29             "%s \n\n" +
30             "Thank you for using Campus Grievance Redressal System. \n\n" +
31             "Best Regards, \n\n" +
32             "CGRS Team",
33             grievanceSubject,
34             status,
35             remarks != null && !remarks.isEmpty() ? "Remarks: " + remarks : ""
36         );
37     }
38 }
```

5.1.6 Frontend Template Module

(Thymeleaf HTML page)



```
1  <!DOCTYPE html>
2  <html xmlns:th="http://www.thymeleaf.org">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Grievance Details - CGRS</title>
7      <link rel="stylesheet" th:href="@{/css/style.css}">
8  </head>
9  <body>
10     <nav class="navbar">
11         <div class="container">
12             <div class="nav-brand">
13                 <a th:href="@{/dashboard}">CGRS</a>
14             </div>
15             <div class="nav-menu">
16                 <a th:href="@{/dashboard}">Dashboard</a>
17                 <a th:href="@{/logout}">Logout</a>
18             </div>
19         </div>
20     </nav>
21
22     <main class="main-content">
23         <div class="container">
24             <div class="grievance-header">
25                 <h1>Grievance Details</h1>
26                 <a th:href="@{/dashboard}" class="btn btn-secondary">Back to Dashboard</a>
27             </div>
28
29             <div class="grievance-details">
30                 <div class="detail-card">
31                     <div class="detail-header">
32                         <h2>Text: ${grievance.subject}</h2>
33                         <span class="status-badge" th:classappend="status- + ${grievance.status.name().toLowerCase()}"
34                             th:text="${grievance.status.displayName}>Status</span>
35                     </div>
36                 </div>
37             </div>
38         </div>
39     </main>
40 </body>
41 </html>
```

5.2 Integration of Modules/ Files

1. User Authentication Module -This module handles user registration, login validation, password encryption, and role-based access control for faculties and administrators. It is integrated with the database to verify user credentials and grant appropriate access.

2. Complaint Submission Module -This module allows faculties to submit grievances online by filling out complaint details. The submitted grievance is validated, categorized, and stored in the database for further processing.

3. Complaint Management Module -This module is integrated with the admin dashboard, allowing administrators to view submitted complaints, assign categories, update grievance status, and resolve issues efficiently.

4. Complaint Tracking Module -This module enables faculties to track the current status of their grievances such as **Pending**, **In Progress**, or **Resolved**, ensuring transparency in the grievance redressal process.

5. Database Integration Module -The system is connected with MySQL for storing user records, grievance details, complaint categories, and notification logs. Proper entity relationships ensure efficient data management.

6. Frontend Integration Module -The frontend is developed using **HTML**, **CSS**, and **Thymeleaf** templates. It is integrated with the backend to dynamically display user-specific data and application responses.

7. Email Notification Module -The email module is integrated with backend services to automatically send notifications when a grievance is submitted or when the complaint status is updated.

8. Security Integration Module -The security module integrates authentication and authorization using **Spring Security**, ensuring secure access to system resources.

Integrated Workflow

Faculty → Login → Submit Complaint → Database Storage → Admin Review → Status Update → Email Notification → Faculty Tracking

This integration ensures that all modules work together efficiently to provide a reliable and user-friendly grievance management system.

5.3 Screenshots/ Reports/ Dataset details (if required)

1. Login Page:

- Main welcome screen of the application. This page provides secure login access for faculties and administrators

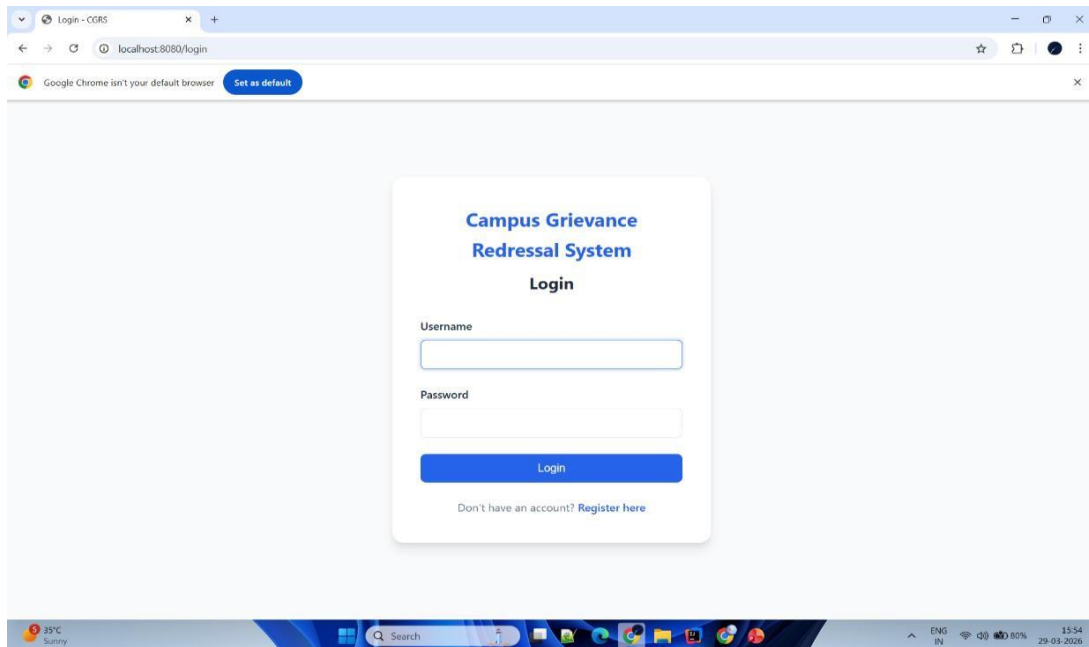


Fig 14. Login Page

2. User Registration Page :

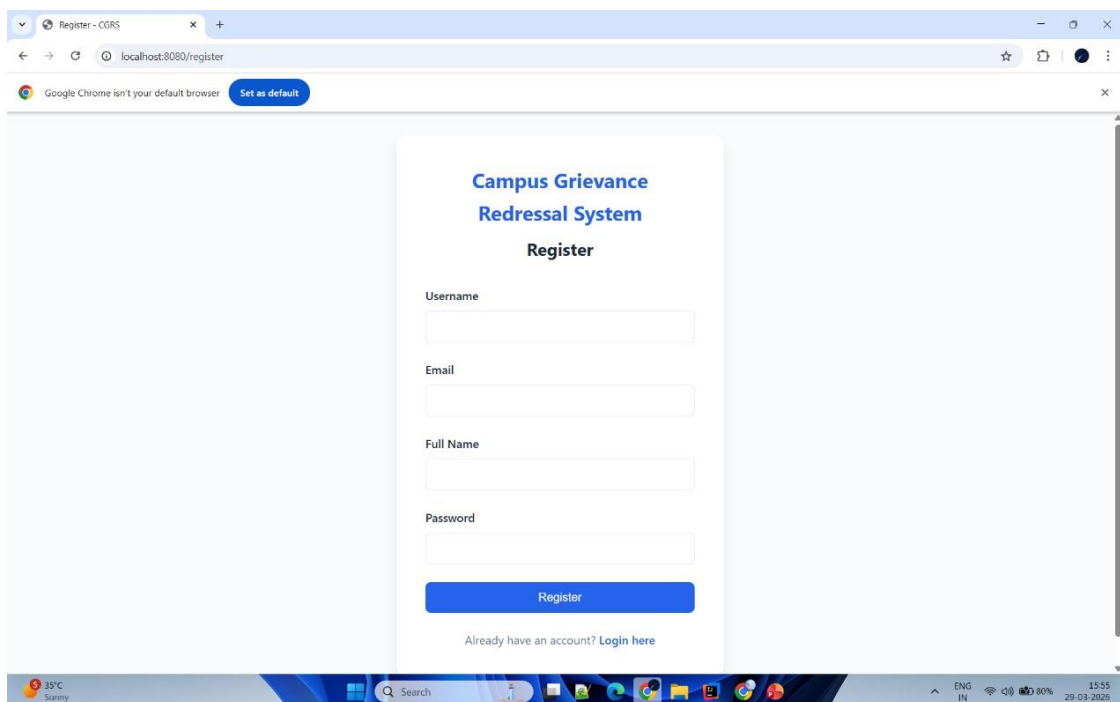


Fig 15. User Registration Page

3. Grievance Submission Page:

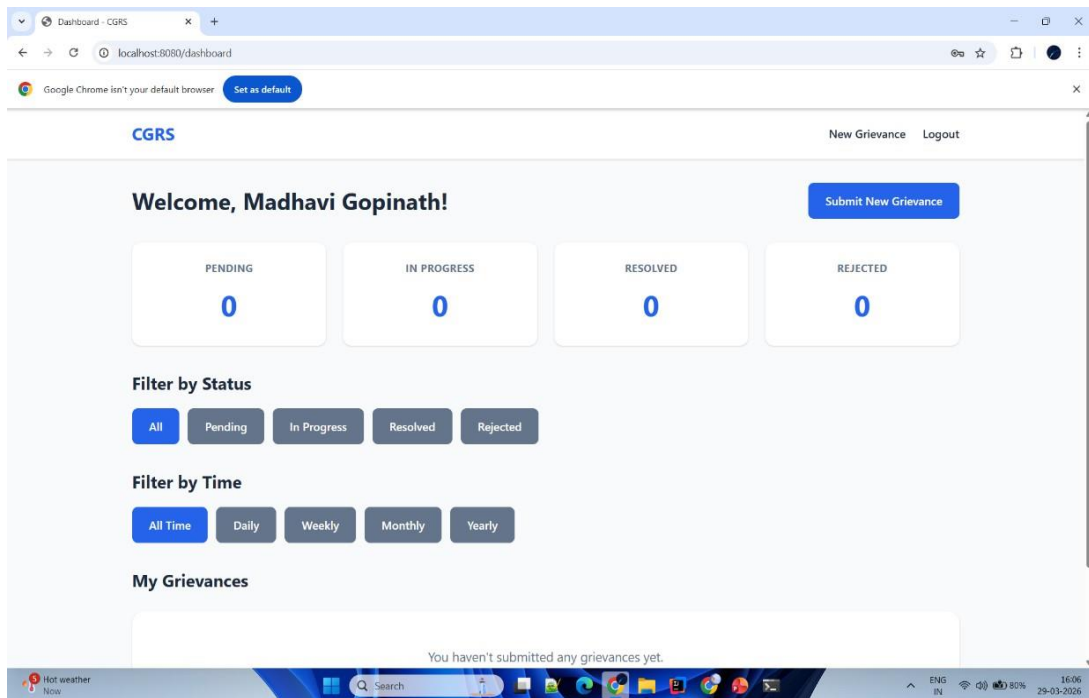
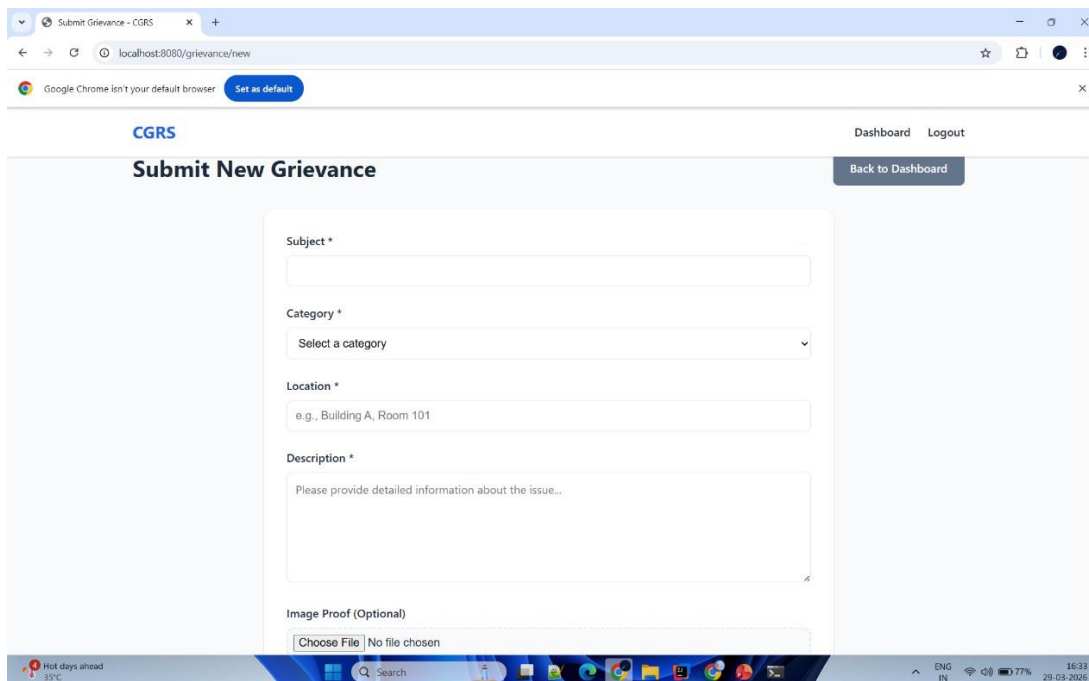


Fig 16. Grievance Submission Page

4. Grievance Details Submission Page :



The top screenshot shows the 'Submit New Grievance' page. The 'Category' dropdown is open, showing a list of categories: 'Select a category', 'Electrical Issues', 'Computer Hardware', 'Projectors & Classroom Equipment', 'Furniture Issues', 'Library & Faculty Room', and 'Water Supply & Facilities'. The 'Image Proof (Optional)' section is visible below the category dropdown.

The bottom screenshot shows the same page with the 'Category' dropdown closed. The 'Location' field is filled with 'e.g., Building A, Room 101'. The 'Description' field has a placeholder text: 'Please provide detailed information about the issue...'. The 'Image Proof (Optional)' section shows a 'Choose File' button and a 'No file chosen' status. Below this, there is a note: 'Upload an image as proof of the issue (Max 10MB). Supported formats: JPG, PNG, GIF'. At the bottom, there are 'Submit Grievance' and 'Cancel' buttons.

Fig 17. Grievance Details Submission Page

5. Admin Dashboard:

The screenshot shows the 'Admin Dashboard' for CGRS. At the top, there are links for 'Dashboard', 'Reports', and 'Logout'. The main section displays four cards representing grievance status: 'PENDING' (4), 'IN PROGRESS' (3), 'RESOLVED' (0), and 'REJECTED' (0). Below these cards are two filter sections: 'Filter by Status' with buttons for 'All', 'Pending', 'In Progress', 'Resolved', and 'Rejected'; and 'Filter by Time' with buttons for 'All Time', 'Daily', 'Weekly', 'Monthly', and 'Yearly'. At the bottom, a table displays a grievance entry: 'Fan is not working' by 'Sonal Khede' in the 'Electrical Issues' category, located in 'MMT 01' on '28 Apr 2026 20:03'. The status is 'Pending', and there is a 'Manage' button next to it.

Fig 18. Admin Dashboard

6. Admin Grievance Management Page :

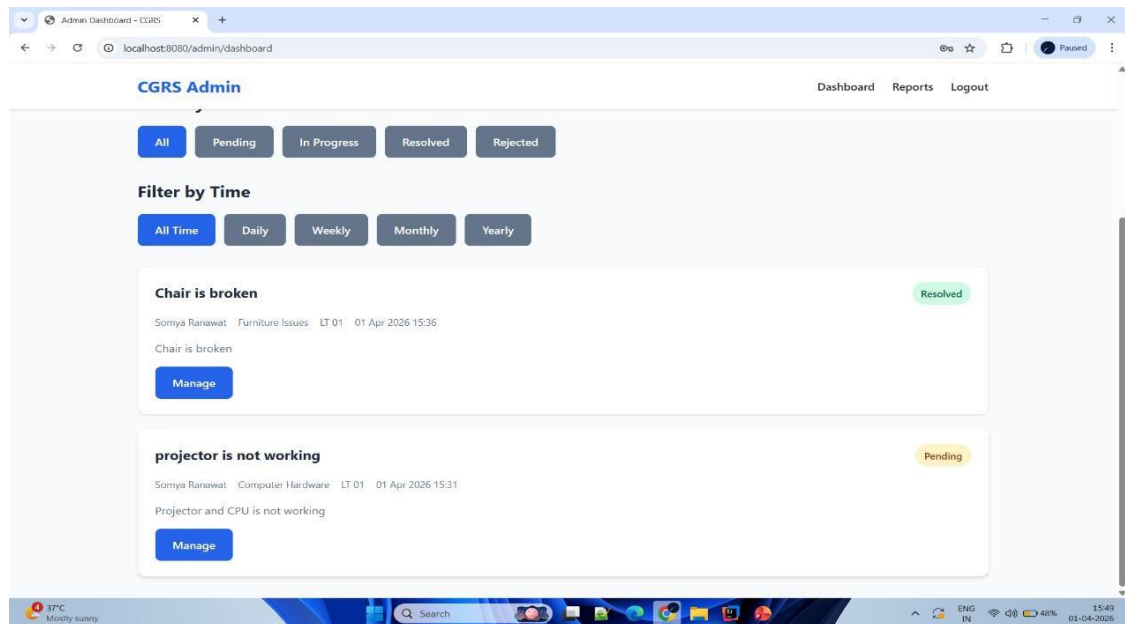


Fig 19. Admin Grievance Management Page

7. Grievance Status Update :

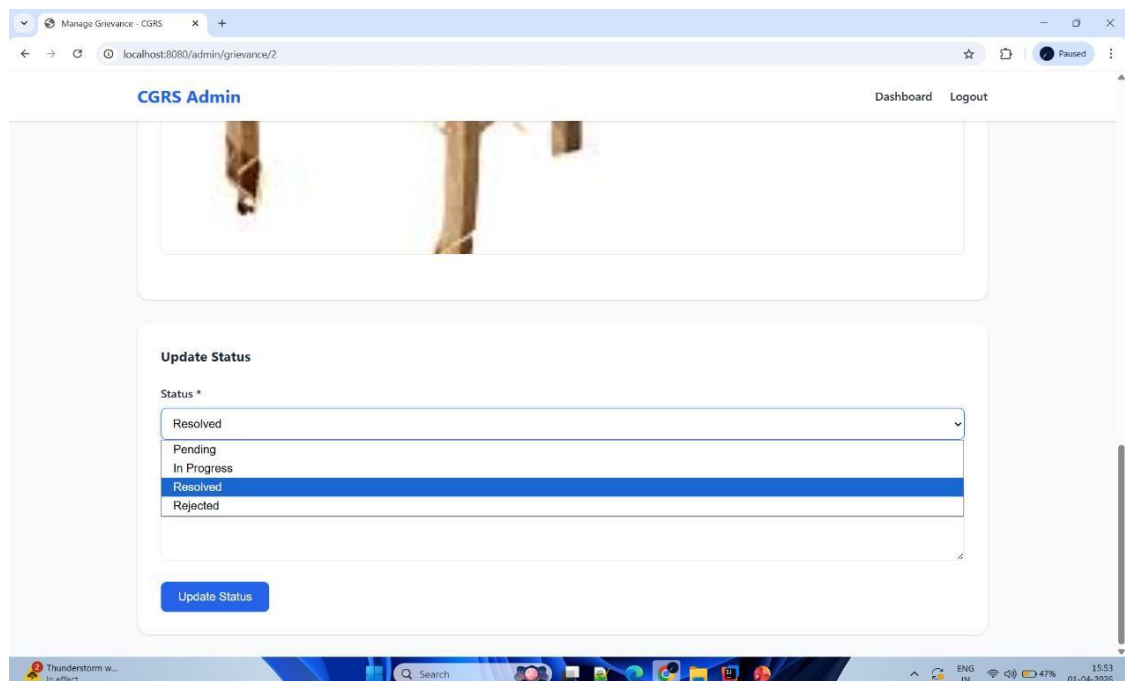


Fig 20. Grievance Status Update Page

8. Report & Analytics:

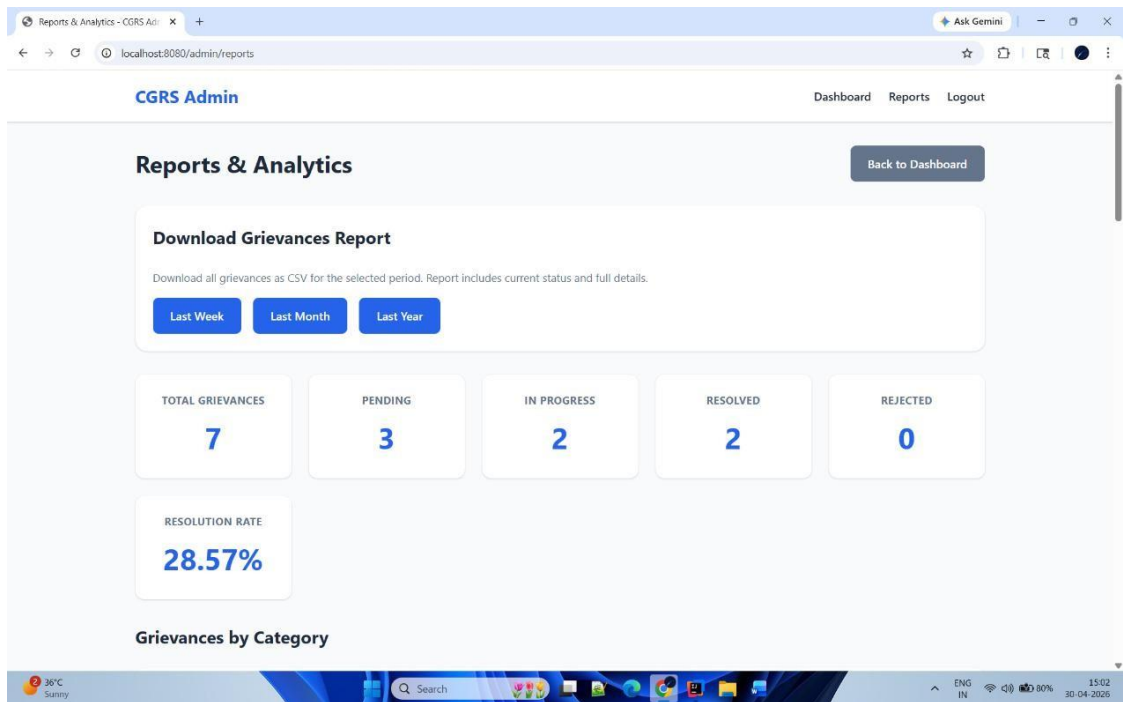


Fig 21. Report & Analytics Dashboard

CHAPTER 6

SYSTEM TESTING

6.1 Test Case Design

Test case design is an essential part of software development used to verify the functionality, security, and performance of the *Campus Grievance Redressal System (CGRS)*. In this project, test cases were designed to evaluate major modules such as user authentication, grievance submission, complaint tracking, admin management, and email notifications. The testing process includes unit testing, integration testing, system testing, and acceptance testing to ensure that the system works accurately, securely, and efficiently under different conditions.

6.1.1 Unit Testing

Unit testing focuses on verifying the functionality of individual modules separately.

Test Case ID	Module	Description	Input	Expected Output	Status
UT01	User Authentication	Validate login credentials	Valid username/password	Login successful	Pass
UT02	User Authentication	Invalid login attempt	Wrong credentials	Error message displayed	Pass
UT03	Complaint Submission	Submit grievance	Complaint details entered	Complaint saved successfully	Pass
UT04	Complaint Tracking	View complaint status	Complaint ID	Status displayed correctly	Pass
UT05	Status Update Module	Update grievance status	Status changed	Status updated successfully	Pass
UT06	Email Notification	Send confirmation email	Complaint submitted	Email delivered successfully	Pass
UT07	Database Module	Store grievance data	New complaint	Record inserted in database	Pass

Table 2. Unit Testing

6.1.2 Integration Testing

Integration testing verifies communication between connected modules.

Test Case ID	Modules	Description	Input	Expected Output	Status
IT01	Login + Database	User authentication	Login request	User verified	Pass
IT02	Complaint Form + Database	Save grievance	Complaint submission	Data stored successfully	Pass
IT03	Admin Dashboard + Database	View complaints	Admin request	Complaint list displayed	Pass
IT04	Status Update + Email	Update grievance status	Status changed	Email notification sent	Pass
IT05	Frontend + Backend	Page request	User action	Correct page rendered	Pass

Table 3. Integration Testing

6.1.3 System Testing

System testing checks the behaviour of the complete integrated application.

Test Case ID	Description	Input	Expected Output	Status
ST01	Complete complaint submission flow	Faculty submits grievance	Complaint saved and confirmation displayed	Pass
ST02	Complaint tracking flow	Faculty checks status	Correct status shown	Pass
ST03	Admin complaint management	Admin updates complaint	Status updated successfully	Pass
ST04	Email notification flow	Complaint resolved	Resolution email sent	Pass
ST05	Security validation	Unauthorized access attempt	Access denied	Pass

Table 4. System Testing

6.1.4 Acceptance Testing

Acceptance testing checks whether the system meets user expectations.

Test Case ID	Scenario	Expected Result	Status
AT01	Faculty login	Login successful	Pass
AT02	Submit grievance	Complaint submitted	Pass
AT03	Track complaint	Correct status visible	Pass
AT04	Admin manages complaint	Complaint updated	Pass
AT05	Email notification	User receives email	Pass
AT06	User interface usability	Easy navigation	Pass

Table 5. Acceptance Testing

6.2 Specific System Testing

Specific system testing focuses on critical system features.

Test Case ID	Feature	Description	Expected Output	Status
SST01	Authentication	Role-based login	Correct dashboard opens	Pass
SST02	Complaint Submission	New complaint registration	Complaint recorded	Pass
SST03	Complaint Tracking	Status monitoring	Accurate status shown	Pass
SST04	Admin Update	Change complaint status	Database updated	Pass
SST05	Email Service	Send notification	Email sent successfully	Pass

Table 6. Specific System Testing

6.3 Test Reports

The designed test cases for unit testing, integration testing, system testing, acceptance testing, and specific system testing were executed successfully. The results indicate that all modules function correctly both individually and in integration.

The system successfully handled secure login authentication, complaint submission, grievance tracking, complaint management, database storage, and automated email notifications. The frontend interface developed using **HTML**, **CSS**, and **Thymeleaf** interacted smoothly with the backend developed using **Spring Boot**, while **MySQL** ensured reliable data storage.

All test cases passed successfully under normal and edge-case scenarios, confirming the **robustness, reliability, security, and overall effectiveness** of the *Campus Grievance Redressal System*.

CHAPTER 7

CONCLUSION OF THE PROJECT

7.1 Results

1. The developed *Campus Grievance Redressal System (CGRS)* was successfully designed, implemented, and tested under practical usage conditions.
2. The system provided a centralized platform for faculties to submit grievances online efficiently, replacing the traditional manual complaint submission process.
3. Faculties were able to submit complaints under different categories such as academic issues, infrastructure problems, library concerns, and other campus-related services.
4. The grievance details were successfully stored and managed in the MySQL database, ensuring data consistency and reliable retrieval.
5. The authentication module correctly validated users and provided role-based access for faculties and administrators.
6. Faculties were able to track the status of submitted grievances in real time, improving transparency in the complaint resolution process.
7. The admin dashboard successfully displayed all submitted grievances and enabled administrators to review, categorize, and update complaint status effectively.
8. The complaint lifecycle management system accurately handled status transitions such as **Pending**, **In Progress**, and **Resolved**.
9. The integrated **Email Notification Module** successfully sent automated notifications to users regarding complaint submission confirmation and grievance status updates.
10. The frontend developed using **HTML, CSS and Thymeleaf** provided a responsive and user-friendly interface.
11. Backend processing using **Spring Boot** efficiently handled business logic, request processing, and database interactions.
12. The system demonstrated reliable performance, secure data handling, and smooth integration between frontend, backend, database, and notification services.
13. Overall, the developed system proved to be an efficient, scalable, and practical solution for grievance management in educational institutions.

7.2 Conclusion

The *Campus Grievance Redressal System (CGRS)* is an effective and efficient web-based application developed to digitalize the grievance handling process within an educational institution. The project successfully addresses the limitations of the traditional manual complaint system by providing a transparent, organized, and user-friendly platform for grievance submission, tracking, and resolution.

The system enhances communication between faculties and administrators by ensuring that complaints are recorded systematically, monitored regularly, and resolved in a timely manner. Features such as secure user authentication, grievance categorization, status tracking, administrative management, and automated email notifications contribute to improving the overall grievance redressal process.

The use of modern technologies such as **Spring Boot**, **Thymeleaf**, **HTML**, **CSS** and **MySQL** made the system robust, scalable, and easy to maintain. Through proper system analysis, design, implementation, and testing, the project has been successfully completed and demonstrates how technology can be effectively used to improve campus services.

In conclusion, the *Campus Grievance Redressal System* fulfills its intended objectives and provides a practical solution for efficient grievance management in colleges and universities.

7.3 Limitations of the project

1. The current system is developed as a **web-based application only** and does not provide a dedicated mobile application.
2. The system depends on a stable internet connection for accessing the platform and receiving email notifications.
3. The notification system currently supports **email communication only**, while SMS and push notifications are not integrated.
4. Complaint categorization and prioritization are manually managed by administrators, which may increase workload in large institutions.
5. The system does not currently provide advanced analytics or dashboards for complaint trend analysis and reporting.
6. Multi-level approval workflows involving department heads or committees are not implemented in the current version.
7. The system is designed primarily for educational institutions and may require modification for use in other sectors.
8. The current version does not include multilingual support, limiting accessibility for diverse user groups.

7.4 Future Work

The *Campus Grievance Redressal System (CGRS)* can be further enhanced by adding new features and improving existing functionalities to make the system more efficient, scalable, and user-friendly.

The following improvements can be considered in future versions of the project:

1. A dedicated mobile application can be developed for Android and iOS platforms to improve accessibility.
2. Additional notification services such as SMS alerts and push notifications can be integrated for real-time communication.
3. An analytics dashboard can be developed for monitoring complaint trends, department-wise statistics, and resolution efficiency.
4. AI-based complaint categorization can be implemented to automatically classify grievances based on complaint content.
5. Multi-level administrative workflows can be introduced for structured grievance approval and escalation.
6. Multi-language support can be added to make the platform accessible to a wider user base.
7. Advanced security features such as two-factor authentication (2FA) and audit logging can be integrated.
8. Cloud deployment can be implemented to improve scalability, performance, and accessibility of the system.

With these future enhancements, the system can evolve into a more comprehensive and intelligent grievance management platform suitable for large-scale institutional use.

7.5 Lessons Learned

The development of the *Campus Grievance Redressal System (CGRS)* provided valuable technical and practical learning experiences throughout the project lifecycle. This project helped in understanding the complete software development process, from requirement analysis and system design to implementation, testing, and deployment.

The key lessons learned during the project are as follows:

1. Gained practical experience in developing a complete full-stack web application using Spring Boot, Thymeleaf, and MySQL.
2. Improved understanding of backend development, including controller handling, service layer implementation, and database integration.
3. Learned how server-side template rendering works using Thymeleaf for dynamic content generation.

4. Acquired practical knowledge of relational database design, table relationships, and CRUD operations.
5. Developed experience in implementing authentication, authorization, and role-based access control.
6. Learned how automated email notification services can be integrated into web applications.
7. Improved frontend development skills using HTML, CSS, and Thymeleaf for responsive UI design.
8. Understood the importance of software testing, debugging, and validation for reliable system performance.
9. Improved teamwork, project planning, and documentation skills during the software development lifecycle.
10. Gained confidence in solving real-world institutional problems through software engineering principles and practical implementation.

Overall, this project served as a strong learning experience by combining academic knowledge with practical implementation, improving both technical expertise and professional development.

REFERENCES

- [1] Spring Boot, “Spring Boot Documentation,” Available:
<https://spring.io/projects/spring-boot>
- [2] Thymeleaf, “Thymeleaf Official Documentation,” Available:
<https://www.thymeleaf.org/documentation.html>
- [3] MySQL, “MySQL Documentation,” Available: <https://dev.mysql.com/doc/>
- [4] IntelliJ IDEA, “IntelliJ IDEA Documentation,” Available:
<https://www.jetbrains.com/idea/documentation/>
- [5] Hibernate, “Hibernate ORM Documentation,” Available:
<https://hibernate.org/orm/documentation/>
- [6] Java, “Java Platform, Standard Edition Documentation,” Available:
<https://docs.oracle.com/javase/>
- [7] HTML, “HTML Standard Documentation,” Available:
<https://developer.mozilla.org/en-US/docs/Web/HTML>
- [8] CSS, “CSS Documentation,” Available: <https://developer.mozilla.org/en-US/docs/Web/CSS>
- [9] Oracle Corporation, “MySQL Reference Manual,” Available:
<https://dev.mysql.com/doc/refman/8.0/en/>
- [10] GitHub Documentation, “GitHub Developer Documentation.” Available:
<https://docs.github.com/>

APPENDICES

Appendix A: Software Components

This appendix provides detailed information about the software components used in the development of the **Campus Grievance Redressal System (CGRS)**. The system integrates backend services, frontend templates, database management, and email notification services to provide an efficient grievance handling platform.

1. Spring Boot Framework

Spring Boot is the core framework used for backend development of the system. It provides a robust platform for building scalable and secure web applications.

Key Features:

- Rapid application development
- Built-in dependency management
- Embedded server support
- REST API development support
- Easy integration with databases and mail services

Role in the System:

- Processes user requests
- Manages authentication and authorization
- Controls grievance workflow
- Integrates email notification services

2. Thymeleaf Template Engine

Thymeleaf is used for creating dynamic web pages by integrating backend data directly into HTML templates.

Key Features:

- Dynamic HTML rendering
- Seamless Spring Boot integration
- Easy template management
- Secure server-side rendering

Role in the System:

- Generates login pages
- Displays faculty dashboard
- Displays admin dashboard
- Renders complaint submission forms
- Shows grievance tracking information dynamically

3. MySQL Database

MySQL is used for storing all project-related data.

Key Features:

- Structured relational database
- Efficient query execution
- Data integrity support
- Primary and foreign key relationships
- Secure data storage

Role in the System:

- Stores user information
- Stores grievance records
- Stores complaint categories
- Maintains grievance status updates
- Supports reporting and retrieval

4. Email Notification Module

The system includes an automated email notification service that sends updates related to grievance activities.

Key Features:

- Automatic email alerts
- Complaint submission confirmation
- Status update notifications
- SMTP-based mail service integration

Role in the System:

- Notifies faculties after complaint submission
- Sends status change updates
- Improves communication between faculties and administration
- Enhances transparency in grievance handling

5. Frontend Technologies

The user interface is developed using **HTML**, **CSS** along with **Thymeleaf** for dynamic rendering.

Role in the System:

- Provides responsive user interface
- Complaint submission forms
- Dashboard pages
- Complaint tracking screens
- Admin management panel

Appendix B: System Module Architecture

This appendix describes the architecture and major modules used in the *Campus Grievance Redressal System*.

The system is divided into multiple functional modules:

- User Authentication Module
- FacultyDashboard Module
- Grievance Submission Module
- Complaint Tracking Module
- Admin Dashboard Module
- Category Management Module
- Email Notification Module

1. User Authentication Module**Description:**

This module handles registration, login, and role-based access control.

Working Logic:

- User enters credentials
- System validates user information
- Role is identified (Faculty/Admin)
- Access granted to respective dashboard

Functions:

- Registration
- Login
- Logout
- Password encryption
- Role validation

2. Grievance Submission Module**Description:**

Allows faculties to submit complaints digitally.

Working Logic:

- Faculty fills complaint form
- Selects category
- Complaint stored in database
- Initial status set to **Pending**
- Confirmation email sent automatically

Functions:

- Submit complaint
- Upload details
- Category selection
- Save grievance record
- Trigger notification

3. Complaint Tracking Module**Description:**

Allows faculties to monitor complaint progress.

Working Logic:

- Faculty logs in

- Views submitted grievances
- Tracks complaint status:
 - Pending
 - In Progress
 - Resolved

Functions:

- View complaint history
- Track current status
- Receive updates

4. Admin Dashboard Module

Description:

This module is used by administrators to manage submitted grievances, monitor complaint progress, update grievance status, and generate grievance reports for analysis and record keeping.

Working Logic:

- Admin logs into the dashboard
- Views all submitted grievances
- Reviews complaint details
- Updates grievance status (Pending / In Progress / Resolved)
- Sends status updates to faculties through email notifications
- Generates grievance reports based on different time periods and complaint status

Functions:

- View all grievances
- Review complaint details
- Update complaint status
- Resolve complaints
- Generate grievance reports
- Download reports (Weekly / Monthly / Yearly)
- Filter reports by status (Pending / Resolved / In Progress / Rejected)
- Send notification updates to Faculties